

Word Vectors and word2vec

Lecture note 5: word2vec + manage experiments

CS 20: TensorFlow for Deep Learning Research

cs20.stanford.edu

Prepared by Chip Huyen (chiphuyen@cs.stanford.edu)

We've built several simple models and they all take only a few minutes to train. To build and train more complex models, we'd need a little bit more tools. In this lecture, we will go over the concept of model base, variable sharing, model sharing, and how to manage our experiments. We will use word2vec as an example to showcase this.

Word2vec

Most of you are probably already familiar with word embedding and understand the importance of a model like word2vec. For those who aren't, [Stanford CS 224N's lecture on word vectors](#) is a great resource. When you're at it, it might be a good idea to check out the following two papers:

If the term is new to you, the intuition is worth a paragraph. A word embedding is a dense vector of a few hundred numbers standing in for a word, learned so that words appearing in similar contexts end up near one another. That is the whole trick: nothing about the spelling of a word is used, only the company it keeps. Everything later in this note follows from that one idea, so it is worth sitting with before reading on. It is also worth being clear about what the vectors are not. They are not a dictionary, they carry no definition, and they have no notion of truth; two words that are used interchangeably in text will sit close together whether or not they mean the same thing, which is why antonyms are often near neighbours. What the geometry captures is substitutability in context, and most of the surprising behaviour of these models, good and bad, follows from that distinction.

[Distributed Representations of Words and Phrases and their Compositionality](#) (Mikolov et al., 2013)

[Efficient Estimation of Word Representations in Vector Space](#) (Mikolov et al., 2013)

At a high level, we need to find an efficient way to represent textual data (in this case, words) so that we can use this representation to solve natural language tasks. Word embeddings form the backbone in the solutions to many tasks such as language modeling, machine translation, sentiment analysis, etc.

Created by a team of researchers led by Tomas Mikolov, word2vec is a group of models that are used to produce word embeddings. There are two main models used in word2vec: skip-gram and CBOW.

Skip-gram vs CBOW (Continuous Bag-of-Words)

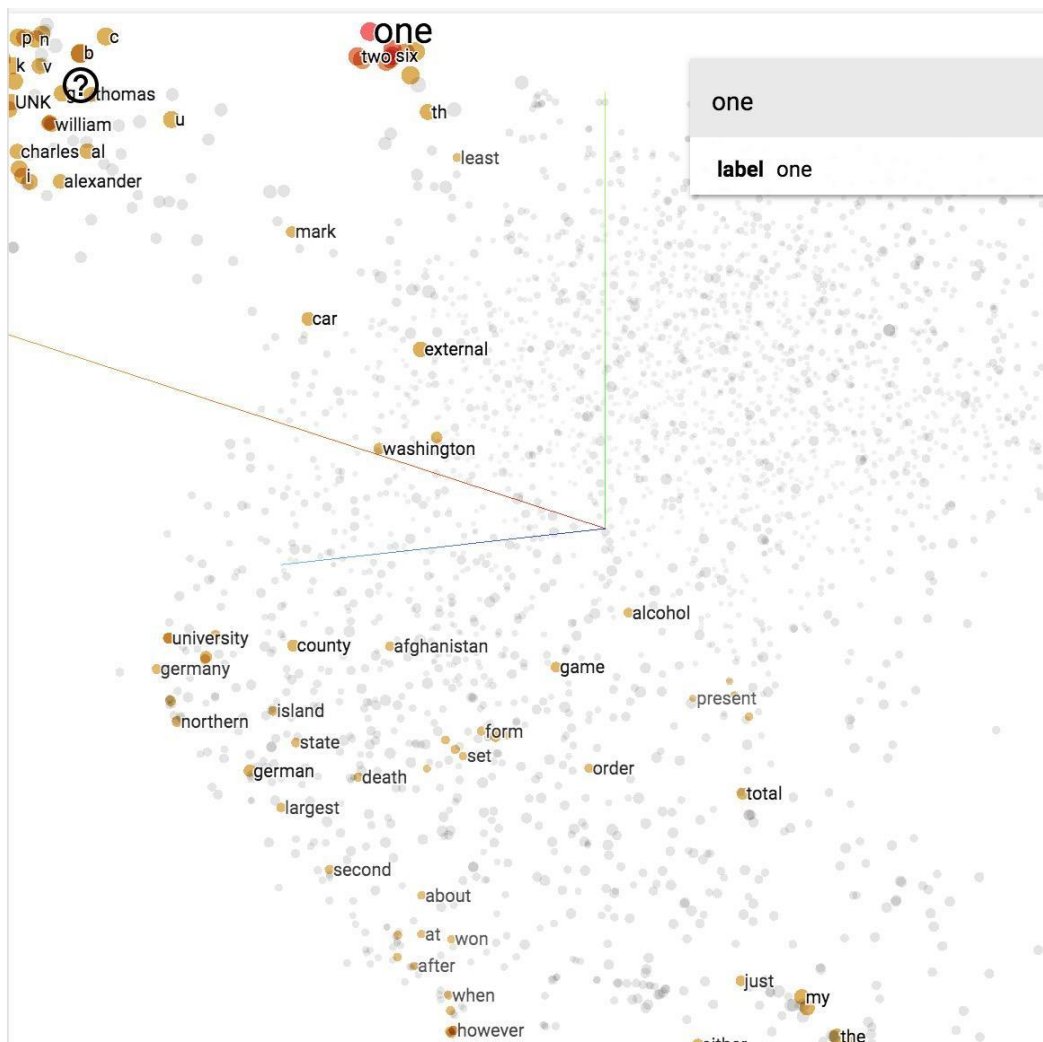
Algorithmically, these models are similar, **except** that CBOW predicts center words **from** context words, **while** the skip-gram does the inverse **and** predicts source context-words **from** the center words. **For** example, **if** we have the sentence: `"The quick brown fox jumps"`, **then** CBOW tries to predict `"brown"` **from** `"the"`, `"quick"`, `"fox"`, **and** `"jumps"`, **while** skip-gram tries to predict `"the"`, `"quick"`, `"fox"`, **and** `"jumps"` **from** `"brown"`.

Statistically it has the effect that CBOW smoothes over a lot of the distributional information (**by** treating an entire context **as** one observation). **For** the most part, **this** turns **out** to be a useful thing **for** smaller datasets. **However**, skip-gram treats each context-target pair **as** a new observation, **and this** tends to **do** better **when** we have larger datasets.

In this lecture, we are going to build word2vec, the skip-gram model. In this model, to get the vector representations of words, we train a simple neural network with a single hidden layer to perform a certain task, but then we don't use that neural network for the task we train it on. Instead, we care about the weights of the hidden layer. These weights are actually the "word vectors", or "embedding matrix" that we're trying to learn.

The certain, fake task we're going to train our model on is predicting the neighboring (context) words given the center word. Given a specific word in a sentence (the center word), look at the words nearby and pick one at random. The network is going to tell us the probability for every word in our vocabulary of being a neighbor to a specific word. Chris McCormick wrote a [tutorial](#) that explains the skip-gram model wonderfully if you want more details.

Vector representations of words visualized with t-SNE projected on a 3D space, using TensorBoard.



Softmax, Negative Sampling, and Noise Contrastive Estimation

To get the distribution of the possible neighboring words, in theory, we often use softmax. Softmax maps arbitrary values x_i to a probability distribution p_i . In this case, $\text{softmax}(x_i)$ is the probability that x_i is a neighboring word of a specific word we are considering.

Softmax is the obvious choice here and also the impractical one, which is the tension the next two sections resolve.

$$\text{softmax}(x_i) = \exp(x_i) / \sum_i \exp(x_i)$$

However, the normalization term in the denominator requires us to perform $\exp$ on all words in the dictionary and sum the results up, which could be millions of words. Even if you disregard uncommon words, a natural language model doesn't perform well unless you consider at least tens of thousands of the most common words. The normalization term causes softmax to be computationally prohibitive.

There are two main approaches to circumvent this bottleneck: hierarchical softmax and sample-based softmax. Mikolov et al. have shown in their paper [*Distributed Representations of Words and Phrases and their Compositionality*](#) that for training the skip-gram model, negative sample results in faster training and better vector representations for frequent words, compared to more complex hierarchical softmax.

Negative sampling, as the name suggests, belongs to the family of sample-based approaches. This family also includes importance sampling and target sampling. Negative sampling is actually a simplified model of an approach called Noise Contrastive Estimation (NCE), e.g. negative sampling makes certain assumption about the number of noise samples to generate -- let's call it k -- and the distribution of noise samples -- let's call it Q -- such that $kQ(w) = 1$ to simplify computation. For more details, please see Sebastian Rudder's [*On word embeddings - Part 2: Approximating the Softmax*](#) and Chris Dyer's [*Notes on Noise Contrastive Estimation and Negative Sampling*](#).

While negative sampling is useful for the learning word embeddings, it doesn't have the theoretical guarantee that its derivative tends towards the gradient of the softmax function. NCE, on the contrary, offers this guarantee as the number of noise samples increases. [Mnih and Teh \(2012\)](#) reported that 25 noise samples are sufficient to match the performance of the regular softmax, with an expected speed-up factor of about 45. For this reason, in this example, we will be using NCE.

Note that sample-based approaches, whether it's negative sampling or NCE, are only useful at training time -- during inference, the full softmax still needs to be computed to obtain a normalized probability.

Dataset

text8 is the first 200 MB of cleaned text of the English Wikipedia dump on Mar. 3, 2006 (whose link is no longer available). We use text that has already been pre-processed because it takes a lot of time to process the raw text and we'd rather use the time in this class to focus on TensorFlow. We can download the dataset [here](#). The file word_utils.py on our GitHub repo has a script to download and read the text

100MB is not enough to train really good word embeddings, but enough to see some interesting relations. There are 17,005,207 tokens if you count tokens by splitting the text by blank space. For better results, you should use the dataset `fi19` of the first 10^9 bytes of the Wikipedia dump, as described on [Matt Mahoney's website](#).

Implementing word2vec

Phase 1: Assemble the graph

1. Create dataset and generate samples from them

Input is the center word and output is the neighboring (context) word. Instead of feeding words into our model, we create a dictionary of the most common words, and feed the indices of those words. For example, if the center word is the 1000^{th} word in the vocabulary, we input the number 999.

Each sample input is a scalar, so `BATCH_SIZE` of sample inputs have shape `[BATCH_SIZE]`. Similarly, `BATCH_SIZE` of sample outputs have shape `[BATCH_SIZE, 1]`.

2. Define the weight (in this case, embedding matrix)

Each row corresponds to the representation vector of one word. If one word is represented with a vector of size `EMBED_SIZE`, then the embedding matrix will have shape `[VOCAB_SIZE, EMBED_SIZE]`. We initialize the embedding matrix to value from a random distribution. In this case, let's choose uniform distribution.

3. Inference (compute the forward path of the graph)

Our goal is to get the vector representations of words in our dictionary. Remember that the `embed_matrix` has dimension `VOCAB_SIZE x EMBED_SIZE`, with each row of the embedding matrix corresponds to the vector representation of the word at that index. So to get the representation of all the center words in the batch, we get the slice of all corresponding rows in the embedding matrix. TensorFlow provides a convenient method to do so.